

Escuela Politécnica Nacional

Métodos Numéricos

Nombre: Erick Cuichan

Curso: GR1CC

Tarea 05

Conjunto de ejercicios

1. Sea $f(x) = -x^3 - \cos x$ y $p_0 = -1$. use el método de Newton y de la Secante para encontrar p_2 . ¿Se podría usar $p_0 = 0$

```
In [14]: import numpy as np
from scipy.optimize import newton, bisect

import matplotlib.pyplot as plt

def funcion(x):
    return -x**3 - np.cos(x)

def funcion_derivada(x):
    return -3 * x**2 + np.sin(x)

inicio = -1
raiz_newton_1 = newton(funcion, fprime=funcion_derivada, x0=inicio)
print(f"Raíz con el método de Newton: {raiz_newton_1}")

raiz_secante_1 = newton(funcion, x0=-1, x1=0)
print(f"Raíz con el método de la Secante: {raiz_secante_1}")

x_inicial_1 = 0
x_inicial_2 = 1
print("Usando x0 = 0 como punto inicial:")

try:
    raiz_newton_2 = newton(funcion, fprime=funcion_derivada, x0=x_inicial_1)
    print(f"Raíz encontrada con Newton: {raiz_newton_2}")
except RuntimeError:
    print(f"El método de Newton no converge con x0 = {x_inicial_1}")

try:
    raiz_secante_2 = newton(funcion, x0=x_inicial_1, x1=x_inicial_2)
    print(f"Raíz encontrada con el método de la Secante: {raiz_secante_2}")
except RuntimeError:
    print(f"El método de la Secante no converge con x0 = {x_inicial_1} y x1 = {x_inicial_2}")
```

Raíz con el método de Newton: -0.8654740331016144
Raíz con el método de la Secante: -0.8654740331016144
Usando $x_0 = 0$ como punto inicial:
El método de Newton no converge con $x_0 = 0$
El método de la Secante no converge con $x_0 = 0$ y $x_1 = 1$

2. Encuentre soluciones precisas dentro de 10^{-4} para los siguientes problemas.

```
In [7]: def ecuacion_a(t):
        return t**3 - 2*t**2 - 5

        def ecuacion_b(t):
            return t**3 + 3*t**2 - 1

        def ecuacion_c(t):
            return t - np.cos(t)

        def ecuacion_d(t):
            return t - 0.8 - 0.2*np.sin(t)

        try:
            raiz_a = newton(ecuacion_a, x0=1, x1=4, tol=1e-4)

            raiz_b = newton(ecuacion_b, x0=-3, x1=-2, tol=1e-4)

            raiz_c = newton(ecuacion_c, x0=0, x1=np.pi/2, tol=1e-4)

            raiz_d = newton(ecuacion_d, x0=0, x1=np.pi/2, tol=1e-4)

        except RuntimeError as error:
            print(f"El método no converge: {error}")
```

a. $x^3 - 2x^2 - 5 = 0$, $[1, 4]$

```
In [8]: print(f"a) Cero aproximado con método de la secante: {raiz_a}")
```

a) Cero aproximado con método de la secante: 2.6906474478837734

b. $x^3 + 3x^2 - 1 = 0$, $[-3, -2]$

```
In [ ]: print(f"b) Cero aproximado con método de la secante: {raiz_b}")
```

b) Cero aproximado con método de la secante: -2.879385194736809

c. $x - \cos x = 0$, $\left[0, \frac{\pi}{2}\right]$

```
In [ ]: print(f"c) Cero aproximado con método de la secante: {raiz_c}")
```

c) Cero aproximado con método de la secante: 0.739085133034638

d. $x - 0.8 - 0.2 \sin x = 0$, $\left[0, \frac{\pi}{2}\right]$

```
In [ ]: print(f"d) Cero aproximado con método de la secante: {raiz_d}")
```

d) Cero aproximado con método de la secante: 0.9643338835706312

3. Use los 2 métodos en esta sección para encontrar las soluciones dentro de 10^{-5} para los siguientes problemas

```
In [ ]: def ecuacion_f(x):
        return 3 * x - np.exp(x)

def ecuacion_g(x):
    return 2 * x + 3 * np.cos(x) - np.exp(x)

def derivada_f(x):
    return 3 - np.exp(x)

def derivada_g(x):
    return 2 - 3 * np.sin(x) - np.exp(x)

try:

    raiz_newton_f = newton(ecuacion_f, fprime=derivada_f, x0=2, tol=1e-5)

    raiz_newton_g = newton(ecuacion_g, fprime=derivada_g, x0=2, tol=1e-5)

    raiz_secante_f = newton(ecuacion_f, x0=1, x1=2, tol=1e-5)

    raiz_secante_g = newton(ecuacion_g, x0=1, x1=2, tol=1e-5)

except RuntimeError as error:
    print(f"El método no converge: {error}")
```

Raíz obtenida para g(x): 1.2397146979752176

Cálculo de raíces con el método de la Secante:

Raíz obtenida para f(x): 1.5121345517620621

Raíz obtenida para g(x): 1.2397146979752534

$$a. \quad 3x - e^x = 0, \quad 1 \leq x \leq 2$$

```
In [6]: print("Cálculo de raíces con el método de Newton:")
        print(f"Raíz obtenida para f(x): {raiz_newton_f}")

        print("\nCálculo de raíces con el método de la Secante:")
        print(f"Raíz obtenida para f(x): {raiz_secante_f}")
```

Cálculo de raíces con el método de Newton:

Raíz obtenida para f(x): 1.5121345516685927

Cálculo de raíces con el método de la Secante:

Raíz obtenida para f(x): 1.5121345517620621

$$2x + 3 \cos x - e^x = 0, \quad 1 \leq x \leq 2$$

```
In [7]: print("Cálculo de raíces con el método de Newton:")
        print(f"Raíz obtenida para g(x): {raiz_newton_g}")

        print("\nCálculo de raíces con el método de la Secante:")
        print(f"Raíz obtenida para g(x): {raiz_secante_g}")
```

Cálculo de raíces con el método de Newton:
Raíz obtenida para $g(x)$: 1.2397146979752176

Cálculo de raíces con el método de la Secante:
Raíz obtenida para $g(x)$: 1.2397146979752534

4. El polinomio de cuarto grado

El polinomio de cuarto grado está dado por:

$$f(x) = 230x^4 + 18x^3 + 9x^2 - 221x - 9$$

Este polinomio tiene dos ceros reales, uno en el intervalo $[-1, 0]$ y otro en $[0, 1]$. Se pide aproximar estos ceros dentro de una tolerancia de 10^{-6} utilizando los siguientes métodos:

```
In [15]: def f(x):
          return 230*x**4 + 18*x**3 + 9*x**2 - 221*x - 9

          def fprime(x):
              return 920*x**3 + 54*x**2 + 18*x - 221

          # MÉTODO DE LA SECANTE"
          # Primer intervalo [-1, 0]
          resultado_secante1 = newton(f, x0=-1, x1=0, tol=1e-6)
          # Segundo intervalo [0, 1]
          resultado_secante2 = newton(f, x0=0, x1=1, tol=1e-6)
          # Modificando el intervalo para encontrar la otra raíz
          resultado_secante3 = newton(f, x0=0.2, x1=1, tol=1e-6)

          # MÉTODO DE NEWTON

          # Primer punto medio (-0.5)
          resultado_newton1 = newton(f, fprime=fprime, x0=-0.5, tol=1e-6)
          # Segundo punto medio (0.5)
          resultado_newton2 = newton(f, fprime=fprime, x0=0.5, tol=1e-6)
          # Ajustando el punto medio para la otra raíz
          resultado_newton3 = newton(f, fprime=fprime, x0=0.6, tol=1e-6)
```

a. Método de la secante, usando los extremos como las estimaciones iniciales.

```
In [16]: print("\nMÉTODO DE LA SECANTE")
          print(f"Resultado cuando el intervalo es [-1, 0]: {resultado_secante1}")
          print(f"Resultado cuando el intervalo es [0, 1]: {resultado_secante2}")
          print(f"Resultado con intervalo ajustado [0.2, 1]: {resultado_secante3}")
```

MÉTODO DE LA SECANTE

Resultado cuando el intervalo es $[-1, 0]$: -0.04065928831575775

Resultado cuando el intervalo es $[0, 1]$: -0.04065928831557162

Resultado con intervalo ajustado $[0.2, 1]$: -0.0406592883141202

b. Método de Newton, empleando el punto medio como estimación inicial.

```
In [18]: print("\nMÉTODO DE NEWTON")
          print(f"Resultado cuando el punto medio es -0.5: {resultado_newton1}")
```

```
print(f"Resultado cuando el punto medio es 0.5: {resultado_newton2}")
print(f"Resultado con otro punto medio (0.6): {resultado_newton3}")
```

MÉTODO DE NEWTON

Resultado cuando el punto medio es -0.5: -0.04065928831575899

Resultado cuando el punto medio es 0.5: -0.040659288315758865

Resultado con otro punto medio (0.6): 0.9623984187505416

5. La función $f(x) = \tan(\pi x) - 6$ tiene un cero en $\frac{1}{\pi} \arctan 6 \approx 0.447431543$

Sea $p_0 = 0$ y $p_1 = 0.48$, y use 10 iteraciones en cada uno de los siguientes métodos para aproximar esta raíz.

¿Cuál método es más eficaz y por qué?

a. Método de bisección

b. Método de Newton

c. Método de la secante

```
In [15]: def f(x):
          return np.tan(np.pi * x) - 6

def fprime(x):
    return np.pi / (np.cos(np.pi * x))**2

p0 = 0
p1 = 0.48
pm = 0.24
iteraciones = 10

# a) Método de La BISECCIÓN

try:
    raiz_bisect = bisect(f, p0, p1, maxiter=iteraciones)
    print(f"Raíz con bisección: {raiz_bisect}")
except RuntimeError as e:
    print(f"No converge con bisección: {e}")

# b) Método de NEWTON

print("\nMÉTODO DE NEWTON ")
try:
    raiz_newton = newton(f, fprime=fprime, x0=pm, maxiter=iteraciones)
    print(f"Raíz con Newton: {raiz_newton}")
except RuntimeError as e:
    print(f"No converge con Newton: {e}")

# c) Método de La SECANTE

print("\n MÉTODO DE LA SECANTE ")
try:
    raiz_secante = newton(f, x0=p0, x1=p1, maxiter=iteraciones)
    print(f"Raíz con secante: {raiz_secante}")
except RuntimeError as e:
```

```

    print(f"No converge con secante: {e}")

# Conclusión

print("\nConclusión:")
print("Ninguno de los métodos converge dentro de 10 iteraciones,")
print("ya que el número de iteraciones máximo es insuficiente para alcanzar la raíz real,")
print("que se encuentra aproximadamente en  $x \approx 0.44743$ .")

```

No converge con bisección: Failed to converge after 10 iterations.

MÉTODO DE NEWTON

Raíz con Newton: 5.447431543288746

MÉTODO DE LA SECANTE

No converge con secante: Failed to converge after 10 iterations, value is -3694.358600967476.

Conclusión:

Ninguno de los métodos converge dentro de 10 iteraciones, ya que el número de iteraciones máximo es insuficiente para alcanzar la raíz real, que se encuentra aproximadamente en $x \approx 0.44743$.

6. La función descrita por $f(x) = \ln(x^2 + 1) - e^{0.4x} \cos \pi x$ tiene un número infinito de ceros.

- Determine dentro de 10^{-6} , el único cero negativo
- Determine dentro de 10^{-6} , los cuatro ceros positivos más pequeños.
- Determine una aproximación razonable para encontrar el n -ésimo cero positivo más pequeño de f
- Use la parte c) para determinar, dentro de 10^{-6} , el vigesimoquinto cero positivo más pequeño de f

```

In [17]: def f(x):
    return np.log(x**2 + 1) - np.exp(0.4 * x) * np.cos(np.pi * x)

def fprime(x):
    return (2 * x) / (x**2 + 1) - 0.4 * np.exp(0.4 * x) * np.cos(np.pi * x) + np.exp(0.4 * x) * np.sin(np.pi * x)

x0, x1 = -1, 0
tolerancia = 1e-6
raiz_neg = newton(f, x0=x0, x1=x1, tol=tolerancia)
print(f"El cero negativo aproximado usando secante es  $x \approx \{raiz\_neg\}$ ")

raiz_1 = newton(f, x0=0, x1=0.5, tol=tolerancia)
raiz_2 = newton(f, x0=1, x1=1.5, tol=tolerancia)
raiz_3 = newton(f, x0=2, x1=2.5, tol=tolerancia)
raiz_4 = newton(f, x0=3, x1=4, tol=tolerancia)
print(f"Los cuatro ceros positivos más pequeños son:\n"
      f" $x_1 \approx \{raiz\_1\}$ \n"
      f" $x_2 \approx \{raiz\_2\}$ \n")

```

```
f"x3 ≈ {raiz_3}\n"  
f"x4 ≈ {raiz_4}")
```

El cero negativo aproximado usando secante es $x \approx -0.4341430472857305$

Los cuatro ceros positivos más pequeños son:

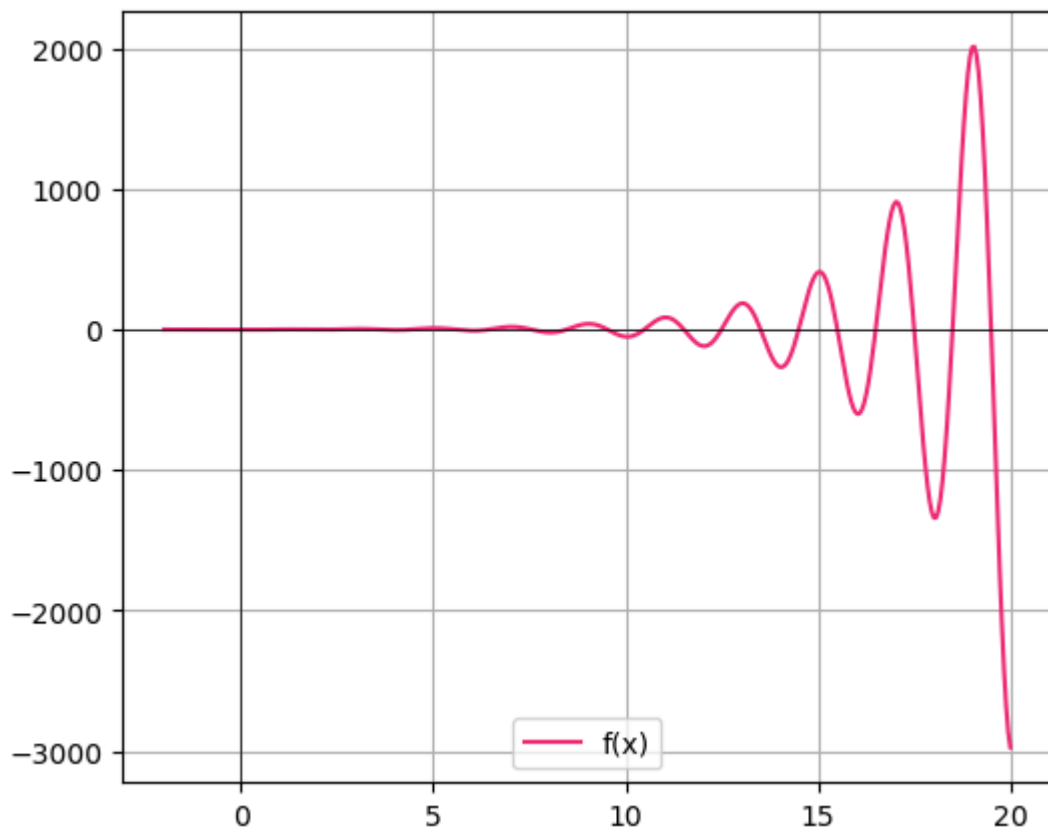
$x_1 \approx 0.4506567478891953$

$x_2 \approx 1.7447380533688277$

$x_3 \approx 2.2383197950401326$

$x_4 \approx 3.7090412013396516$

```
In [18]: x = np.linspace(-2, 20, 400)  
y = np.log(x**2 + 1) - np.exp(0.4 * x) * np.cos(np.pi * x)  
plt.figure(figsize=(6, 5))  
plt.plot(x, y, color="#F02975", label="f(x)")  
plt.axhline(0, color='black', linewidth=0.5)  
plt.axvline(0, color='black', linewidth=0.5)  
plt.legend()  
plt.grid(True)  
plt.show()  
  
roots = []  
for n in range(1, 21):  
    p0 = n - 0.5  
    ans = newton(f, x0=p0, fprime=fprime, tol=1e-6)  
    roots.append((n, ans))  
for n, root in roots:  
    print(f"Raíz aproximada: p = {root} con n = {n}")  
  
n = 25  
p0 = n - 0.5  
ans = newton(f, x0=p0, fprime=fprime, tol=1e-6)  
print(f"Raíz aproximada: p = {ans} con n = {n}")
```



Raíz aproximada: $p = 0.45065674789059323$ con $n = 1$
 Raíz aproximada: $p = 1.7447380533683496$ con $n = 2$
 Raíz aproximada: $p = 2.2383197950741383$ con $n = 3$
 Raíz aproximada: $p = 3.709041201375952$ con $n = 4$
 Raíz aproximada: $p = 4.322648959394466$ con $n = 5$
 Raíz aproximada: $p = 5.619935330897348$ con $n = 6$
 Raíz aproximada: $p = 6.406933614179465$ con $n = 7$
 Raíz aproximada: $p = 7.5632105298048415$ con $n = 8$
 Raíz aproximada: $p = 8.453480976980412$ con $n = 9$
 Raíz aproximada: $p = 9.531833533481178$ con $n = 10$
 Raíz aproximada: $p = 10.477305887176216$ con $n = 11$
 Raíz aproximada: $p = 11.515570723125352$ con $n = 12$
 Raíz aproximada: $p = 12.489106412447065$ con $n = 13$
 Raíz aproximada: $p = 13.507471405108609$ con $n = 14$
 Raíz aproximada: $p = 14.494831050995868$ con $n = 15$
 Raíz aproximada: $p = 15.503539134886033$ con $n = 16$
 Raíz aproximada: $p = 16.497568337549417$ con $n = 17$
 Raíz aproximada: $p = 17.50166147499906$ con $n = 18$
 Raíz aproximada: $p = 18.49886352880028$ con $n = 19$
 Raíz aproximada: $p = 19.500774932317764$ con $n = 20$
 Raíz aproximada: $p = 24.499887047445956$ con $n = 25$

7. La función $f(x) = x^{\frac{1}{3}}$ tiene raíz en $x = 0$. Usando el punto de inicio de $x = 1$ y $p_0 = 5, p_1 = 0.5$ para el método de secante, compare los resultados de los métodos de la secante y de newton.

```
In [19]: def f(x):
          return np.cbrt(x)

          def fprime(x):
              return (1/3) * np.power(x, -2/3)

          try:
              resultado_newton = newton(f, fprime=fprime, x0=1, maxiter=50)
              print(f"Resultado con Newton (x0=1): {resultado_newton}")
          except Exception as e:
              print(f"No converge con Newton: {e}")

          try:
              resultado_secante = newton(f, x0=5, x1=0.5, maxiter=50)
              print(f"Resultado con Secante (p0=5, p1=0.5): {resultado_secante}")
          except Exception as e:
              print(f"No converge con Secante: {e}")
```

No converge con Newton: Failed to converge after 50 iterations, value is nan.
 No converge con Secante: Failed to converge after 50 iterations, value is 0.15125956067017968.

C:\Users\cuich\AppData\Local\Temp\ipykernel_41072\3014065403.py:5: RuntimeWarning:
 g: invalid value encountered in power
 return (1/3) * np.power(x, -2/3)